AMITY UNIVERSITY
UTTAR PRADESH

LAB RECORD

BACHELOR OF TECHNOLOGY

B.Tech. CS&E Semester (V)

(Academic Session – 2021 - 22)

Course Title	**: Analysis and Design of Algorithms**
Course Code	**: CSE303**
Enrollment No.	**: A7605220041**
Name of Student	**: Nandini Tiwari**
Date of Submission	**:**
Signature of Student	**:**
Grade/Marks Obtained	**:**

Department of Computer Science & Engineering

Amity School of Engineering & Technology

Amity University, Lucknow Campus

Department of Computer Science & Engineering

Amity School of Engineering & Technology

Amity University, Lucknow Campus

Index

Sno.	Lab Question	Date	Signature
1	Implement recursive Binary Search and Linear Search. Determine the time required to search an element. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.		
2	Sort a given set of elements using the Quicksort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.		
3	Implement 0/1 Knapsack problem using Greedy Programming.		
4	Implement 0/1 Knapsack problem using Dynamic Programming.		
5	From a given starting node in a digraph, print all the nodes reachable by using BFS/DFS method.		
6	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal'salgorithm.		

	(b) Find Minimum Cost Spanning Tree of a given undirected graph using Prim'salgorithm.		
7	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.		
8	Consider the problem of N queen on an (NxN) chessboard. Two queens are said to attack each other if they are on the same row, column, or diagonal. Implements backtracking algorithm to solve the problem i.e. place N nonattacking queens on the board.		
9	Implement Knapsack Problem based on Backtracking algorithm.		
10	Implement Traveling Salesman problem based on Branch and Bound technique.		
	OPEN ENDED PROGRAM		
1	Develop a Hamiltonian Path in an undirected graph is a path that visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a Hamiltonian Path such that there is an edge (in graph) from the last vertex to the first vertex of the Hamiltonian Path. Develop a program to implement the solution of Travelling Salesman Problem by considering the Hamiltonian cycle approach.		
2	Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, "abc", "abg", "bdf", "aeg", '"acefg", .. etc are subsequences of "abcdefg". So a string of length n has 2n different possible subsequences.		
3	A road network can be considered as a graph with positive weights. The nodes represent road junctions and each edge of the graph is associated with a road segment between two junctions. The weight of an edge may correspond to the length of the associated road segment, the time needed to traverse the segment or the cost of traversing the segment. Using directed edges it is also possible to model one-way streets. Such graphs are special in the sense that some edges are more important than others for long distance travel (e.g. highways). This property has been formalized using the notion of highway dimension. There are a great number of algorithms that exploit this property and are therefore able to compute the shortest path a lot quicker than would be possible on general		

	graphs. Develop a program to find the shortest path from each node to solve the road network problem. Shortest path		
4	There are a number of agents and a number of tasks. Any agent can be assigned to perform any task, incurring so me cost that may vary depending on the agenttask assignment. It is required to perform all tasks by assigning exactly one agent to each task and exactly one task to each agent in such a way that the total cost of the assignment is minimized. If the numbers of agents and tasks are equal and the total cost of the assignment for all tasks is equal to the sum of the costs for each agent (or the sum of the costs for each task, which is the same thing in this case), then the problem is called the linear assignment problem. Develop a program to solve the job assignment problem.		
5	Suppose that we are designing a program to simulate the storage and search in a dictionary. Words appear with different frequencies, however, and it may be the case that a frequently used word such as "the" appears far from the root while a rarely used word such as "conscientiousness" appears near the root. We want words that occur frequently in the text to be placed nearer to the root. Moreover, there may be words in the dictionary for which there is no definition. Write a program to organize an optimal binary search tree that simulates the storage and search of words in a dictionary.		

PROGRAM 1

Implement recursive Binary Search and Linear Search. Determine the time required to search an element. Repeat the experiment for different values of n, the number ofelements in the list to be searched and plot a graph of the time taken versus n.

Explanation:

Functions for linear search and binary search have been developed. But, as binary search requires sorted elements, a function for sorting the elements also has been included in this program.

To compute the time taken by the algorithm, we make use of built-in data type clock_t which is available in the header file time.h. Two variables of type clock_t are required to get the starting time and ending time of the algorithm. The built-in function clock() gives the current time of the system in terms of number of clock-ticks. So, to compute the time elapsed for the working of an algorithm, the different between starting time and ending time has to be considered and it must be divided by CLK_TCK.

As the processors are very fast, the time computed will be negligibly small and hence it may result in zero always. Hence, a delay is enforced wherever the basic operation is expected to execute.

Normally, time complexity can be better understood with a huge value of input size. Hence, instead of reading the array elements from the keyboard, it is suggested to generate random numbers.

Theoretically,

the time complexity of linear search = n

the time complexity of binary search = $\log_2 n$

For a comparative study, it is better to draw the graph as per theoretical time complexity aswell as the actual time computed through the program.

Code:

```
#include<stdio.h>

#include<time.h>

#include<stdlib.h>

#define max 20

int pos;

int binsearch (int,int[],int,int,int);

int linsearch (int,int[],int);

void main()

{ int ch=1; double t; int n,i,a[max],k,op,low,high,pos;

clock_t begin,end,CLK_TCK;

while(ch)

{

printf("\n.......MENU.......\n 1.BinarySearch \n 2.Linear search \n 3.Exit \n");

printf("\n enter your choice\n");

scanf("%d",&op);

switch(op)

{

case 1:printf("\n enter the number of elments\n"); scanf("%d",&n);

printf("\n enter the number of an array in the order \n");

for(i=0;i<n;i++)

scanf("%d",&a[i]);

printf("\n enter the elements to be searched \n");

scanf("%d",&k); low=0;high=n-1; begin=clock();

pos=binsearch(n,a,k,low,high);
```

```
end=clock();
if(pos==-1)
printf("\n\nUnsuccessful search");
else
printf("\n element %d is found at position %d",k,pos+1);
printf("\n Time Taken is %lf CPU1 cycles \n",(end-begin)/CLK_TCK);
break;
case 2:printf("\n enter the number of elements \n");
scanf("%d",&n);
printf("\n enter the elements of an array\n");
for(i=0;i<n;i++)
scanf("%d",&a[i]);
printf("\n enter the element to be searched \n");
scanf("%d",&k);
begin=clock();
pos=linsearch(n,a,k);
end=clock();
if(pos==-1)
printf("\n\n Unsuccessful search");
else
printf("element %d is found at position %d",k,pos+1);
printf("\n Time taken is %lf CPU cycles \n",(end-begin)/CLK_TCK);
break;
default:printf("Invalid choice entered \n");
exit(0);
```

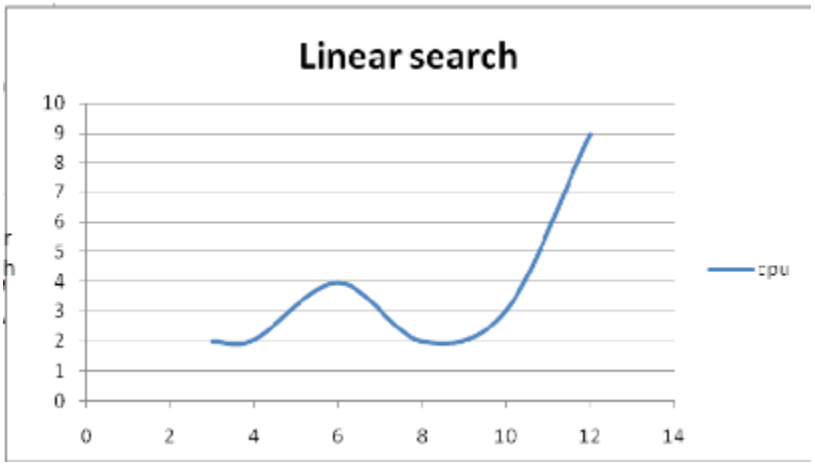
```
}
printf("\n Do you wish to run again(1/0) \n");
scanf("%d",&ch);
}
}
int binsearch(int n,int a[],int k,int low,int high)
{
int mid;
mid=(low+high)/2;
if(low>high)
return -1;
if(k==a[mid])
return(mid);
else
if(k<a[mid])
return binsearch(n,a,k,low,mid-1);
else
return binsearch(n,a,k,mid+1,high);
}
int linsearch(int n,int a[],int k)
{
if(n<0) return -1;
if(k==a[n-1])
return (n-1);
else
```

return linsearch(n-1,a,k);

}

Output

```
.......MENU.......
 1.BinarySearch
 2.Linear search
 3.Exit

 enter your choice
2

 enter the number of elements
3

 enter the elements of an array
98
22
46

 enter the element to be searched
22
element 22 is found at position 2
 Time taken is 1.978022 CPU cycles
```



N	cpu
3	1.978022
4	2.032967
6	3.956044
8	1.978022
10	3.021978
12	8.956044

PROGRAM 2

Sort a given set of elements using the Quicksort method and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>

int quickSort(int a[], int low, int high);

int partition(int a[], int low, int high);


int main() {

    int a[1000], n, i;


    printf("Enter the number of elements: ");

    scanf("%d", &n);


    printf("Entering random values into array:\n");

    for (i = 1; i <= n; i++) {

        a[i] = rand() % 100;

        printf("%d ", a[i]);

    }


    quickSort(a, 1, n);


    printf("\nSorted Elements are:\n");

    for (i = 1; i <= n; i++)
```

```
        printf("%d ", a[i]);

    return 0;
}

int quickSort(int a[], int low, int high) {
    int j;

    if (low < high) {
        j = partition(a, low, high);
        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }

    return 0;
}

int partition(int a[], int low, int high) {
    int key = a[low], i = low + 1, j = high, temp;

    for ( ; ; ) {
        while (key >= a[i] && i < high)
            i++;

        while (key < a[j])
```

```
            j--;

        if (i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        } else {
            temp = a[low];
            a[low] = a[j];
            a[j] = temp;

            return j;
        }
    }
}
```

Output

```
Enter the number of elements: 10
Entering random values into array:
83 86 77 15 93 35 86 92 49 21
Sorted Elements are:
15 21 35 49 77 83 86 86 92 93
```

PROGRAM 3

Implement 0/1 Knapsack problem using Greedy Programming.

```
#include<stdio.h>
int knapsack(int i, int cap, int n, int w[10],int v[10])
{
int x=0, y=0;
if (i==n)
return ((cap<w[n])?0:v[n]);
if(cap<w[ i ])
return knapsack(i+1, cap, n, w, v);
x=knapsack(i+1, cap, n, w, v);
y=knapsack(i+1, cap-w[i], n, w, v) + v[ i ];
return ((x>=y)?x:y);
}
void main()
{
int n, i, w[10],v[10],cap, maxprofit=0;
printf("Enter number of items:");
scanf("%d",&n);
printf("\nEnter respective weights:");
for(i=0;i<n;i++)
scanf("%d",&w[i]);
printf("\nEnter respective profits/values:");
```

```
for(i=0;i<n;i++)
scanf("%d",&v[i]);
printf("\nEnter capacity:");
scanf("%d",&cap);
maxprofit=knapsack(0,cap,n,w,v);
printf("\nMaximum profit is: %d",maxprofit);
}
```

Output

```
Enter number of items:3
Enter respective weights:2 6 1
Enter respective profits/values:5 23 65
Enter capacity:89
Maximum profit is: 22128
```

PROGRAM 4

Implement 0/1 Knapsack problem using Dynamic Programming.

```
#include<stdio.h>

int max(int a, int b) { return (a > b)? a : b; }

int knapSack(int W, int wt[], int val[], int n)

{

   int i, w;

   int K[n+1][W+1];

   for (i = 0; i <= n; i++)

   {

       for (w = 0; w <= W; w++)

       {

           if (i==0 || w==0)

               K[i][w] = 0;

           else if (wt[i-1] <= w)

                K[i][w] = max(val[i-1] + K[i-1][w-wt[i-1]],  K[i-1][w]);

           else

                K[i][w] = K[i-1][w];

       }

   }

   return K[n][W];

}

int main()

{
```

```
    int i, n, val[20], wt[20], W;

    printf("Enter number of items:");
    scanf("%d", &n);

    printf("Enter value and weight of items:\n");
    for(i = 0;i < n; ++i){
     scanf("%d%d", &val[i], &wt[i]);
    }

    printf("Enter size of knapsack:");
    scanf("%d", &W);

    printf("%d", knapSack(W, wt, val, n));
    return 0;
}
```

Output

```
Enter number of items:3
Enter value and weight of items:
100 20
50 10
150 30
Enter size of knapsack:50
250
```

PROGRAM 5

From a given starting node in a digraph, print all the nodes reachable by using BFS/DFS method.

Description:

Breadth-first search (BFS)

It employs the following rules.

1. Start from nodes add it to queue.

2. Visit all neighbors of node s and add it to queue Data structure.

3. Then visit all their neighbors, if not already visited by removing elements from queue.

4. Continue until all nodes visited

5. Visit all nodes in Level order of tree.

Depth First Search (DFS)

It employs the following rules.

1. Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.

2. If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)

3. Repeat Rule 1 and Rule 2 until the stack is empty.

4. Visit all nodes like Preorder traversal of tree.

Solution

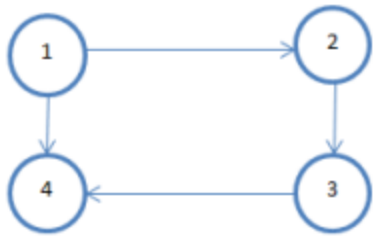
```
#include<stdio.h>

#include<stdlib.h>

int a[50][50], n, visited[50];

int q[20], front = -1,rear = -1;

int s[20], top = -1, count=0;
```

```
void bfs(int v)
{
 int i, cur;
 visited[v] = 1;
 q[++rear] = v;
 while(front!=rear)
 {
 cur = q[++front];
 for(i=1;i<=n;i++)
 {
 if((a[cur][i]==1)&&(visited[i]==0))
 {
 q[++rear] = i;
 visited[i] = 1;
 printf("%d ", i);
 }
 }
 }
}
void dfs(int v)
{
 int i;
 visited[v]=1;
 s[++top] = v;
 for(i=1;i<=n;i++)
```

```
 {
 if(a[v][i] == 1&& visited[i] == 0 )
 {
 printf("%d ", i);
 dfs(i);
 }
 }
}
int main()
{
 int ch, start, i,j;
 printf("\nEnter the number of vertices in graph: ");
 scanf("%d",&n);
 printf("\nEnter the adjacency matrix:\n");
 for(i=1; i<=n; i++)
 {
 for(j=1;j<=n;j++)
 scanf("%d",&a[i][j]);
 }
 for(i=1;i<=n;i++)
 visited[i]=0;
 printf("\nEnter the starting vertex: ");
 scanf("%d",&start);
 printf("\n==>1. BFS: Print all nodes reachable from a given starting node");
 printf("\n==>2. DFS: Print all nodes reachable from a given starting node");
```

```
printf("\n==>3:Exit");
printf("\nEnter your choice: ");
scanf("%d", &ch);
switch(ch)
{
case 1: printf("\nNodes reachable from starting vertex %d are: ", start);
bfs(start);
for(i=1;i<=n;i++)
{
if(visited[i]==0)
printf("\nThe vertex that is not reachable is %d" ,i);
}
break;
case 2: printf("\nNodes reachable from starting vertex %d are:\n",start);
dfs(start);
break;
case 3: exit(0);
default: printf("\nPlease enter valid choice:");
}
}
```

Output



Case 1:

```
Enter the number of vertices in graph: 4
Enter the adjacency matrix:
0 1 0 1
0 0 1 0
0 0 0 1
0 0 0 0
Enter the starting vertex: 1
==>1. BFS: Print all nodes reachable from a given starting node
==>2. DFS: Print all nodes reachable from a given starting node
==>3:Exit
Enter your choice: 1
Nodes reachable from starting vertex 1 are: 2 4 3
```

Case 2:

```
/tmp/rRahdi0THy.o
Enter the number of vertices in graph: 4
Enter the adjacency matrix:
0 1 0 1
0 0 1 0
0 0 0 1
0 0 0 0
Enter the starting vertex: 2
==>1. BFS: Print all nodes reachable from a given starting node
==>2. DFS: Print all nodes reachable from a given starting node
==>3:Exit
Enter your choice: 1
Nodes reachable from starting vertex 2 are: 3 4
The vertex that is not reachable is 1
```

Case 3:

```
/tmp/rRahdi0THy.o
Enter the number of vertices in graph: 4
Enter the adjacency matrix:
0 1 0 1
0 0 1 0
0 0 0 1
0 0 0 0
Enter the starting vertex: 1
==>1. BFS: Print all nodes reachable from a given starting node
==>2. DFS: Print all nodes reachable from a given starting node
==>3:Exit
Enter your choice: 2
Nodes reachable from starting vertex 1 are:
2 3 4
```

Case 4:

```
/tmp/rRahdi0THy.o
Enter the number of vertices in graph: 4
Enter the adjacency matrix:
0 1 0 1
0 0 1 0
0 0 0 1
0 0 0 0
Enter the starting vertex: 2
==>1. BFS: Print all nodes reachable from a given starting node
==>2. DFS: Print all nodes reachable from a given starting node
==>3:Exit
Enter your choice: 2
Nodes reachable from starting vertex 2 are:
3 4
```

PROGRAM 6

(a) **Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal'salgorithm.**

```
#include<stdio.h>
void main()
{
int n, v, u,cost[10][10], parent[10]={0}, i, j;
int count=1, mincost=0, min, a, b;
printf("Enter number of vertices:");
scanf("%d",&n);
printf("\nEnter cost matrix:\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
{
scanf("%d",&cost[i][j]);
if(cost[i][j]==0)
cost[i][j]=999;
}
while(count<n)
{
min=999;
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
if(cost[i][j]<min)
```

```
{
}
while(parent[u])
u=parent[u];
while(parent[v])
v=parent[v];
if(u!=v)
{
min=cost[i][j];
a=u=i;
b=v=j;
count++;
printf("\nEdge(%d, %d) = %d", a, b, min);
mincost+=min;
parent[v]=u;
}
cost[a][b]=cost[b][a]=999;
}
printf("\nMinimum cost = %d", mincost);
}
```

Output

```
/tmp/cKLgCRfBTp.o
Enter number of vertices:7
Enter cost matrix:
0 2 999 3 999 999 999
2 0 9 999 1 4 999
999 9 0 999 999 3 999
3 999 999 0 5 999 7
999 1 999 5 0 999 4
999 4 3 999 999 0 6
999 999 999 7 4 6 0


Edge(2,5)=1

Edge(1,2)=2
Edge(1,4)=3
Edge(3,6)=3
Edge(2,6)=4
Edge(5,7)=4
Minimum cost = 17
```

(b) **Find Minimum Cost Spanning Tree of a given undirected graph using Prim'salgorithm.**

```
#include<stdio.h>
void main()
{
int n, v, u,cost[10][10], visited[10]={0}, i, j;
int count=1, mincost=0, min, a, b;
printf("Enter number of vertices:");
scanf("%d",&n);
printf("\nEnter cost matrix (For infinity, put 999):\n");
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
{
scanf("%d",&cost[i][j]);
if(cost[i][j]==0)
cost[i][j]=999;
}
visited[1]=1;
printf("\nThe edges of spanning tree are: \n");
while(count<n)
{
min=999;
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
```

```
if(cost[i][j]<min)
if(visited[i]==0)
continue;
else
{
}
min=cost[i][j];
a=u=i;
b=v=j;
if(visited[u]==0 || visited[v]==0)
{
count++;
printf("\nEdge(%d, %d) = %d", a, b, min);
mincost+=min;
visited[b]=1;
}
cost[a][b]=cost[b][a]=999;
}
printf("\nMinimum cost = %d", mincost);
}
```

Output

```
/tmp/cKLgCRfBTp.o
Enter number of vertices:7
Enter cost matrix:
0 2 999 3 999 999 999
2 0 9 999 1 4 999
999 9 0 999 999 3 999
3 999 999 0 5 999 7
999 1 999 5 0 999 4
999 4 3 999 999 0 6
999 999 999 7 4 6 0


Edge(2,5)=1

Edge(1,2)=2
Edge(1,4)=3
Edge(3,6)=3
Edge(2,6)=4
Edge(5,7)=4
Minimum cost = 17
```

PROGRAM 7

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

void dijkstra(int n, int v, int cost[10][10],int dist[10])

{

int count, u, i, w, visited[10], min;

for(i=0;i<n;i++)

{

visited[i]=0; dist[i]=cost[v][i];

}

visited[v]=1; dist[v]=1; count=2;

while(count<=n)

{

min=999; for(w=0;w<n;w++)

if((dist[w]<min) && (visited[w]!=1))

{

min=dist[w]; u=w;

}

visited[u]=1; count++; for(w=0;w<n;w++)

if((dist[u]+cost[u][w]<dist[w]) && (visited[w]!=1)) dist[w]=dist[u]+cost[u][w];

}
```

```
}
void main()
{
int n, v, cost[10][10], dist[10], i, j;
printf("Enter number of vertices:"); scanf("%d",&n);
printf("\nEnter cost matrix (for infinity, enter 999):\n"); for(i=0;i<n;i++)
for(j=0;j<n;j++)
scanf("%d",&cost[i][j]);
printf("\nEnter source vertex:"); scanf("%d",&v);
dijkstra(n,v,cost,dist);
printf("\nShortest path from \n"); for(i=0;i<n;i++)
if(i!=v)
printf("\n%d -> %d = %d", v, i, dist[i]);
}
```

Output

```
Enter number of vertices:7
Enter cost matrix (for infinity, enter 999):
0 2 999 3 999 999 999
2 0 9 999 1 4 999
999 9 0 999 999 3 999
3 999 999 0 5 999 7
999 1 999 5 0 999 7
999 4 3 999 999 0 6
999 999 999 7 4 6 0
Enter source vertex:0
Shortest path from

0 -> 1 = 2
0 -> 2 = 9
0 -> 3 = 3
0 -> 4 = 3
0 -> 5 = 6
0 -> 6 = 10
```

PROGRAM 8

Consider the problem of N queen on an (NxN) chessboard. Two queens are said to attack each other if they are on the same row, column, or diagonal. Implements backtracking algorithm to solve the problem i.e. place N nonattacking queens on the board.

```
#include<stdio.h>

#include<math.h>

#include<stdlib.h>

int place(int m[50], int k)

{

int i;

for(i=0;i<k;i++)

if((m[i]==m[k])||(abs(m[i]-m[k])==abs(i-k)))

return 0;

return 1;

}

void display(int m[50],int n)

{
```

```
int i, j;
int s[10][10]={0};
for(i=0;i<n;i++)
s[i][m[i]]=1;
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
if(s[i][j])
printf("Q\t");
else
printf("x\t");
printf("\n");
}
exit(1);
}
void main()
{
int m[50], s[50][50], n, k;
printf("Enter number of Queens:");
scanf("%d",&n);
printf("\nThe solution for the problem is:\n");
n--;
for(m[0]=0, k=0;k>=0; m[k]+=1)
{
while((m[k]<=n) && (!place(m,k))){
```

```
m[k]+=1;
if(m[k]<=n){
if(k==n)
display(m,n+1);
else
{
k--;
k++;
m[k]=-1;
}
}
}
}
}
```

Output

```
Enter number of Queens:4
The solution for the problem is:
x Q x x
x x x Q
Q x x x
x x Q x
```

PROGRAM 9

Implement Knapsack Problem based on Backtracking algorithm

```
knapsack problem using backtracking*/
 /* Variable Description….
 n – Total no. of items available
w[] – Weight of each item
p[] – Profit of each item
m – Maximum Capacity of the Sack
unit[] – Profit of each item per Unit
p[]/w[] x[] – Final list of items put into the Sack
y[] – Intermediate list of selected items
fp – Final Profit
fw – Final Weight
cp – Current Profit
cw – Current Weight  */

#include <stdio.h>
```

```
#define max 10
int w[max],i,j,p[max];
int n,m;
float unit[max];
int y[max],x[max],fp=-1,fw;
void get()
{
printf("\n Enter total number of items: ");
scanf("%d",&n);
printf("\n Enter the Maximum capacity of the Sack: ");
scanf("%d",&m);
for(i=0;i<n;i++)
{
printf("\n Enter the weight of the item # %d : ",i+1);
scanf("%d",&w[i]);
printf("\n Enter the profit of the item # %d : ", i+1);
scanf("%d", &p[i]);
}
}
void show()
{
float s=0.0;
printf("\n\tItem\tWeight\tCost\tUnit Profit\tSelected ");
for(i=0;i<n;i++)
printf("n\t%d\t%d\t%d\t%f\t%d",i+1,w[i],p[i],unit[i],x[i]);
```

```
printf("\n\n The Sack now holds following items : ");
for(i=0;i<n;i++)
if(x[i]==1)
{
printf("%d\t",i+1);
s += (float) p[i] * (float) x[i];
}
printf("\n Maximum Profit: %f ",s);
}

void sort()
{
int t,t1;
float t2;
for(i=0;i<n;i++)
unit[i] = (float) p[i] / (float) w[i];
for(i=0;i<n-1;i++)
{
for(j=i+1;j<n;j++)
{
if(unit[i] < unit[j])
{
t2 = unit[i];
unit[i] = unit[j];
unit[j] = t2;
```

```
t = p[i];
p[i] = p[j];
p[j] = t;
t1 = w[i];
w[i] = w[j];
w[j] =t1;
}
}
}
}
float bound(float cp,float cw,int k)
{
float b = cp;
float c = cw;
for(i=k;i<=n;i++)
{
c = c+w[i];
if( c < m)
b = b +p[i];
else
return (b+(1-(c-m)/ (float)w[i])*p[i]);
}
return b;
}
void knapsack(int k,float cp,float cw)
```

```
{
if(cw+w[k] <= m)
{
y[k] = 1;
if(k <= n)
knapsack(k+1,cp+p[k],cw+w[k]);
if(((cp+p[k]) > fp) && ( k == n))
{
fp = cp+p[k];
fw = cw+w[k];
for(j=0;j<=k;j++)
x[j] = y[j];
}
}
if(bound(cp,cw,k) >= fp)
{
y[k] = 0;
if( k <= n)
knapsack(k+1,cp,cw);
if((cp > fp) && (k == n))
{
fp = cp;
fw = cw;
for(j=0;j<=k;j++)
x[j] = y[j];
```

```
}
}
}
void main()
{
printf("\n\n\n\t\t ******** KNAPSACK PROBLEM ********");
printf("\n\t\t ——————————————————————————");
get();
printf("\n The Sack is arranged in the order…\n");
sort();
knapsack(0,0.0,0.0);
show();
}
```

Output

```
/tmp/UOsHOPLjWK.o
******** KNAPSACK PROBLEM ********
        ______________
 Enter total number of items: 3
 Enter the Maximum capacity of the Sack: 25
 Enter the weight of the item # 1 : 1
 Enter the profit of the item # 1 : 11
 Enter the weight of the item # 2 : 11
 Enter the profit of the item # 2 : 21
 Enter the weight of the item # 3 : 21
 Enter the profit of the item # 3 : 31
 The Sack is arranged in the order…

    Item    Weight  Cost    Unit Profit Selected n  1   1   11  11.000000
        1n  2   11  21  1.909091    0n  3   21  31  1.476190    1

The Sack now holds following items : 1  3
 Maximum Profit: 42.000000
```

PROGRAM 10

Implement Traveling Salesman problem based on Branch and Bound technique.

```
#include <bits/stdc++.h>
```

```
using namespace std;
const int N = 4;
int final_path[N+1];
bool visited[N];

int final_res = INT_MAX;

void copyToFinal(int curr_path[])
{
	for (int i=0; i<N; i++)
		final_path[i] = curr_path[i];
	final_path[N] = curr_path[0];
}
int firstMin(int adj[N][N], int i)
{
	int min = INT_MAX;
	for (int k=0; k<N; k++)
		if (adj[i][k]<min && i != k)
			min = adj[i][k];
	return min;
}

int secondMin(int adj[N][N], int i)
{
	int first = INT_MAX, second = INT_MAX;
```

```
        for (int j=0; j<N; j++)
        {
                if (i == j)
                        continue;

                if (adj[i][j] <= first)
                {
                        second = first;
                        first = adj[i][j];
                }
                else if (adj[i][j] <= second &&
                                adj[i][j] != first)
                        second = adj[i][j];
        }
        return second;
}

void TSPRec(int adj[N][N], int curr_bound, int curr_weight,
                        int level, int curr_path[])
{

        if (level==N)
        {

                if (adj[curr_path[level-1]][curr_path[0]] != 0)
```

```
        {

            int curr_res = curr_weight +
                        adj[curr_path[level-1]][curr_path[0]];



            if (curr_res < final_res)
            {
                copyToFinal(curr_path);
                final_res = curr_res;
            }
        }
        return;
    }



    for (int i=0; i<N; i++)
    {

        if (adj[curr_path[level-1]][i] != 0 &&
            visited[i] == false)
        {
            int temp = curr_bound;
            curr_weight += adj[curr_path[level-1]][i];
```

```
                    if (level==1)
                    curr_bound -= ((firstMin(adj, curr_path[level-1]) +
                                        firstMin(adj, i))/2);
                    else
                    curr_bound -= ((secondMin(adj, curr_path[level-1]) +
                                        firstMin(adj, i))/2);



                    if (curr_bound + curr_weight < final_res)
                    {
                        curr_path[level] = i;
                        visited[i] = true;
                        TSPRec(adj, curr_bound, curr_weight, level+1,
                            curr_path);
                    }
                    curr_weight -= adj[curr_path[level-1]][i];
                    curr_bound = temp;
                    memset(visited, false, sizeof(visited));
                    for (int j=0; j<=level-1; j++)
                        visited[curr_path[j]] = true;
                }
            }
}
void TSP(int adj[N][N])
```

```
{
        int curr_path[N+1];
        int curr_bound = 0;
        memset(curr_path, -1, sizeof(curr_path));
        memset(visited, 0, sizeof(curr_path));

                for (int i=0; i<N; i++)
                curr_bound += (firstMin(adj, i) +
                                        secondMin(adj, i));

                curr_bound = (curr_bound&1)? curr_bound/2 + 1 :
                                                        curr_bound/2;

        visited[0] = true;
        curr_path[0] = 0;



        TSPRec(adj, curr_bound, 0, 1, curr_path);
}

int main()
{
        int adj[N][N] = { {0, 10, 15, 20},
                {10, 0, 35, 25},
                {15, 35, 0, 30},
```

```
        {20, 25, 30, 0}
    };

    TSP(adj);

    printf("Minimum cost : %d\n", final_res);
    printf("Path Taken : ");
    for (int i=0; i<=N; i++)
        printf("%d ", final_path[i]);

    return 0;
}
```

Output

```
/tmp/Jy3Cx82vqS.o
Minimum cost : 80
Path Taken : 0 1 3 2 0
```

OPEN ENDED QUESTIONS

PROGRAM 1

Develop a Hamiltonian Path in an undirected graph is a path that visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a Hamiltonian Path such that there is an edge (in graph) from the last vertex to the first vertex of the Hamiltonian Path. Develop a program to implement the solution of Travelling Salesman Problem by considering the Hamiltonian cycle approach.

```
#include <bits/stdc++.h>

using namespace std;

#define V 4


// implementation of traveling Salesman Problem

int travllingSalesmanProblem(int graph[][V], int s)

{

 // store all vertex apart from source vertex

 vector<int> vertex;

 for (int i = 0; i < V; i++)

 if (i != s)

 vertex.push_back(i);
```

```
 // store minimum weight Hamiltonian Cycle.
 int min_path = INT_MAX;
 do {

 // store current Path weight(cost)
 int current_pathweight = 0;

 // compute current path weight
 int k = s;
 for (int i = 0; i < vertex.size(); i++) {
 current_pathweight += graph[k][vertex[i]];
 k = vertex[i];
 }
 current_pathweight += graph[k][s];
 min_path = min(min_path, current_pathweight);

 } while (next_permutation(vertex.begin(), vertex.end()));

 return min_path;
}

// driver program to test above function
int main()
{
 // matrix representation of graph
```

```
int graph[][V] = { { 0, 10, 15, 20 },
{ 10, 0, 35, 25 },
{ 15, 35, 0, 30 },
{ 20, 25, 30, 0 } };
int s = 0;
cout << travllingSalesmanProblem(graph, s) << endl;
return 0;
}
```

Output

```
/tmp/Jy3Cx8zvqS.o
80
```

PROGRAM 2

Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, “abc”, “abg”, “bdf”, “aeg”, ‘”acefg”, .. etc are subsequences of “abcdefg”. So a string of length n has 2n different possible subsequences. Examples: LCS for input Sequences “ABCDGH” and “AEDFHR” is “ADH” of length 3. LCS for input Sequences “AGGTAB” and “GXTXAYB” is “GTAB” of length 4.

```
#include<bits/stdc++.h>


int max(int a, int b);


int lcs( char *X, char *Y, int m, int n )

{

if (m == 0 || n == 0)

        return 0;

if (X[m-1] == Y[n-1])

        return 1 + lcs(X, Y, m-1, n-1);

else

        return max(lcs(X, Y, m, n-1), lcs(X, Y, m-1, n));
```

```
}

int max(int a, int b)
{
        return (a > b)? a : b;
}


int main()
{
char X[] = "AGGTAB";
char Y[] = "GXTXAYB";

int m = strlen(X);
int n = strlen(Y);

printf("Length of LCS is %d", lcs( X, Y, m, n ) );

return 0;
}
```

Output

```
/tmp/Jy3Cx8zvqS.o
Length of LCS is 4
```

PROGRAM 3

A road network can be considered as a graph with positive weights. The nodes represent road junctions and each edge of the graph is associated with a road segment between two junctions. The weight of an edge may correspond to the length of the associated road segment, the time needed to traverse the segment or the cost of traversing the segment. Using directed edges it is also possible to model one-way streets. Such graphs are special in the sense that some edges are more important than others for long distance travel (e.g. highways). This property has been formalized using the notion of highway dimension. There are a great number of algorithms that exploit this property and are therefore able to compute the shortest path a lot quicker than would be possible on general graphs. Develop a program to find the shortest path from each node to solve the road network problem. Shortest path

```
#include <limits.h>

#include <stdio.h>



#define V 9



int minDistance(int dist[], bool sptSet[])

{


        int min = INT_MAX, min_index;


        for (int v = 0; v < V; v++)

                if (sptSet[v] == false && dist[v] <= min)

                        min = dist[v], min_index = v;
```

```
	return min_index;
}


int printSolution(int dist[], int n)
{
	printf("Vertex Distance from Source\n");
	for (int i = 0; i < V; i++)
		printf("%d \t\t %d\n", i, dist[i]);
}


void dijkstra(int graph[V][V], int src)
{
	int dist[V];
	bool sptSet[V];
	for (int i = 0; i < V; i++)
		dist[i] = INT_MAX, sptSet[i] = false;


	dist[src] = 0;


	for (int count = 0; count < V - 1; count++) {
```

```
            int u = minDistance(dist, sptSet);



            sptSet[u] = true;

            for (int v = 0; v < V; v++)



                if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                    && dist[u] + graph[u][v] < dist[v])
                    dist[v] = dist[u] + graph[u][v];
        }



        printSolution(dist, V);
}



int main()
{

        int graph[V][V] = { { 0, 4, 0, 0, 0, 0, 0, 8, 0 },
                            { 4, 0, 8, 0, 0, 0, 0, 11, 0 },
                            { 0, 8, 0, 7, 0, 4, 0, 0, 2 },
                            { 0, 0, 7, 0, 9, 14, 0, 0, 0 },
```

```
                                        { 0, 0, 0, 9, 0, 10, 0, 0, 0 },
                                        { 0, 0, 4, 14, 10, 0, 2, 0, 0 },
                                        { 0, 0, 0, 0, 0, 2, 0, 1, 6 },
                                        { 8, 11, 0, 0, 0, 0, 1, 0, 7 },
                                        { 0, 0, 2, 0, 0, 0, 6, 7, 0 } };

        dijkstra(graph, 0);

        return 0;
}
```

Output

```
/tmp/nEtLQUbup.o
Vertex Distance from Source
0        0
1        4
2        12
3        19
4        21
5        11
6        9
7        8
8        14
```

PROGRAM 4

There are a number of agents and a number of tasks. Any agent can be assigned to perform any task, incurring some cost that may vary depending on the agent task assignment. It is required to perform all tasks by assigning exactly one agent to each task and exactly one task to each agent in such a way that the total cost of the assignment is minimized. If the numbers of agents and tasks are equal and the total cost of the assignment for all tasks is equal to the sum of the costs for each agent (or the sum of the costs for each task, which is the same thing in this case), then the problem is called the linear assignment problem. Develop a program to solve the job assignment problem.

```
#include<bits/stdc++.h>

using namespace std;



class Solution {

public:


        int cost[31][31]; //cost matrix

        int n, max_match; //n workers and n jobs

        int lx[31], ly[31]; //labels of X and Y parts

        int xy[31]; //xy[x] - vertex that is matched with x,

        int yx[31]; //yx[y] - vertex that is matched with y

        bool S[31], T[31]; //sets S and T in algorithm

        int slack[31]; //as in the algorithm description

        int slackx[31]; //slackx[y] such a vertex, that

        int prev_ious[31]; //array for memorizing alternating p


        void init_labels()
```

```
{
	memset(lx, 0, sizeof(lx));
	memset(ly, 0, sizeof(ly));
	for (int x = 0; x < n; x++)
	for (int y = 0; y < n; y++)
	lx[x] = max(lx[x], cost[x][y]);
}


void update_labels()
{
	int x, y;
	int delta = 99999999; //init delta as infinity
	for (y = 0; y < n; y++) //calculate delta using slack
		if (!T[y])
			delta = min(delta, slack[y]);
	for (x = 0; x < n; x++) //update X labels
		if (S[x])
			lx[x] -= delta;
	for (y = 0; y < n; y++) //update Y labels
		if (T[y])
			ly[y] += delta;
	for (y = 0; y < n; y++) //update slack array
		if (!T[y])
			slack[y] -= delta;
```

```
}



void add_to_tree(int x, int prev_iousx)
{
	S[x] = true; //add x to S
	prev_ious[x] = prev_iousx; //we need this when augmenting
	for (int y = 0; y < n; y++) //update slacks, because we add new vertex to S
		if (lx[x] + ly[y] - cost[x][y] < slack[y])
		{
			slack[y] = lx[x] + ly[y] - cost[x][y];
			slackx[y] = x;
		}
}




void augment() //main function of the algorithm
{
	if (max_match == n) return; //check whether matching is already perfect
	int x, y, root; //just counters and root vertex
	int q[31], wr = 0, rd = 0; //q - queue for bfs, wr,rd - write and read
	//pos in queue
	memset(S, false, sizeof(S)); //init set S
	memset(T, false, sizeof(T)); //init set T
```

```
memset(prev_ious, -1, sizeof(prev_ious)); //init set prev_ious - for the alternating tree

for (x = 0; x < n; x++)
{
	if (xy[x] == -1)
	{
		q[wr++] = root = x;
		prev_ious[x] = -2;
		S[x] = true;
		break;
	}
}

for (y = 0; y < n; y++)
{
	slack[y] = lx[root] + ly[y] - cost[root][y];
	slackx[y] = root;
}

while (true)		{
	while (rd < wr)

	{
		x = q[rd++];
```

```
        for (y = 0; y < n; y++)
                if (cost[x][y] == lx[x] + ly[y] && !T[y])
                {
                        if (yx[y] == -1) break;
                                T[y] = true;
                        q[wr++] = yx[y];

                        add_to_tree(yx[y], x);
                }
        if (y < n)
                break;
}
if (y < n)
        break;

update_labels();

wr = rd = 0;
for (y = 0; y < n; y++)
if (!T[y] && slack[y] == 0)
{
        if (yx[y] == -1)
        {
                x = slackx[y];
                break;
```

```
                }
                else
                {
                        T[y] = true;
                        if (!S[yx[y]])
                        {
                                q[wr++] = yx[y];
                                add_to_tree(yx[y], slackx[y]);
                                        }
                }
        }
        if (y < n) break;
}

if (y < n)
{
        max_match++;

        for (int cx = x, cy = y, ty; cx != -2; cx = prev_ious[cx], cy = ty)
        {
                ty = xy[cx];
                yx[cy] = cx;
                xy[cx] = cy;
        }
        augment();
```

```
        }
}

int hungarian()
{
        int ret = 0;
        max_match = 0;
        memset(xy, -1, sizeof(xy));
        memset(yx, -1, sizeof(yx));
        init_labels();
        augment();

        for (int x = 0; x < n; x++)
                ret += cost[x][xy[x]];

        return ret;
}

int assignmentProblem(int Arr[], int N) {

        n = N;
        for(int i=0; i<n; i++)
                for(int j=0; j<n; j++)
                        cost[i][j] = -1*Arr[i*n+j];
```

```
        int ans = -1 * hungarian();


        return ans;
    }
};


int main()
{
int n=3;
int Arr[3*3]={1500,4000,4500,2000,6000,3500,2000,4000,2500};
Solution ob;
cout<<ob.assignmentProblem(Arr,n)<<endl;
}
```

Output

```
/tmp/nEftLQUbup.o
8500
```

PROGRAM 5

Suppose that we are designing a program to simulate the storage and search in a dictionary. Words appear with different frequencies, however, and it may be the case that a frequently used word such as "the" appears far from the root while a rarely used word such as "conscientiousness" appears near the root. We want words that occur frequently in the text to be placed nearer to the root. Moreover, there may be words in the dictionary for which there is no definition. Write a program to organize an optimal binary search tree that simulates the storage and search of words in a dictionary

```
#include<stdio.h>

#define MAX 10


void main()

{

        char ele[MAX][MAX];

        int w[MAX][MAX], c[MAX][MAX], r[MAX][MAX], p[MAX], q[MAX];

        int temp=0, root, min, min1, n;

        int i,j,k,b;

        printf("Enter the number of elements:");

        scanf("%d",&n);

        printf("\n");

        for(i=1; i <= n; i++)

        {

                printf("Enter the Element of %d:",i);

                scanf("%d",&p[i]);

        }

        printf("\n");
```

```
	for(i=0; i <= n; i++)
	{
		printf("Enter the Probability of %d:",i);
		scanf("%d",&q[i]);
	}
	printf("W\t\tC\t\tR\n");
	for(i=0; i <= n; i++)
	{
		for(j=0; j <= n; j++)
		{
			if(i == j)
			{
				w[i][j] = q[i];
				c[i][j] = 0;
				r[i][j] = 0;
				printf("W[%d][%d]: %d\tC[%d][%d]: %d\tR[%d][%d]:
%d\n",i,j,w[i][j],i,j,c[i][j],i,j,r[i][j]);
			}
		}
	}
	printf("\n");
	for(b=0; b < n; b++)
	{
		for(i=0,j=b+1; j < n+1 && i < n+1; j++,i++)
		{
```

```
                    if(i!=j && i < j)
                    {
                        w[i][j] = p[j] + q[j] + w[i][j-1];


                        min = 30000;
                        for(k = i+1; k <= j; k++)
                        {
                            min1 = c[i][k-1] + c[k][j] + w[i][j];
                            if(min > min1)
                            {
                                min = min1;
                                temp = k;
                            }
                        }
                        c[i][j] = min;
                        r[i][j] = temp;
                    }
                    printf("W[%d][%d]: %d\tC[%d][%d]: %d\tR[%d][%d]:
%d\n",i,j,w[i][j],i,j,c[i][j],i,j,r[i][j]);
                }
                printf("\n");
            }
            printf("Minimum cost = %d\n",c[0][n]);
            root = r[0][n];
            printf("Root  = %d \n",root);  }
```

Output

```
/tmp/GR4YCmNSgp.o
Enter the number of elements:6
Enter the Element of 1:10
Enter the Element of 2:3
Enter the Element of 3:9
Enter the Element of 4:2
Enter the Element of 5:0
Enter the Element of 6:10
Enter the Probability of 0:5
Enter the Probability of 1:6
Enter the Probability of 2:4
Enter the Probability of 3:4
Enter the Probability of 4:3
Enter the Probability of 5:8
Enter the Probability of 6:0
W       C       R
W[0][0]: 5  C[0][0]: 0  R[0][0]: 0
W[1][1]: 6  C[1][1]: 0  R[1][1]: 0
W[2][2]: 4  C[2][2]: 0  R[2][2]: 0
W[3][3]: 4  C[3][3]: 0  R[3][3]: 0
W[4][4]: 3  C[4][4]: 0  R[4][4]: 0
```

```
W        C        R
W[0][0]: 5  C[0][0]: 0  R[0][0]: 0
W[1][1]: 6  C[1][1]: 0  R[1][1]: 0
W[2][2]: 4  C[2][2]: 0  R[2][2]: 0
W[3][3]: 4  C[3][3]: 0  R[3][3]: 0
W[4][4]: 3  C[4][4]: 0  R[4][4]: 0
W[5][5]: 8  C[5][5]: 0  R[5][5]: 0
W[6][6]: 0  C[6][6]: 0  R[6][6]: 0

W[0][1]: 21 C[0][1]: 21 R[0][1]: 1
W[1][2]: 13 C[1][2]: 13 R[1][2]: 2
W[2][3]: 17 C[2][3]: 17 R[2][3]: 3
W[3][4]: 9  C[3][4]: 9  R[3][4]: 4
W[4][5]: 11 C[4][5]: 11 R[4][5]: 5
W[5][6]: 18 C[5][6]: 18 R[5][6]: 6

W[0][2]: 28 C[0][2]: 41 R[0][2]: 1
W[1][3]: 26 C[1][3]: 39 R[1][3]: 3
W[2][4]: 22 C[2][4]: 31 R[2][4]: 3
W[3][5]: 17 C[3][5]: 26 R[3][5]: 5
W[4][6]: 21 C[4][6]: 32 R[4][6]: 6
```

```
W[2][4]: 22 C[2][4]: 31 R[2][4]: 3
W[3][5]: 17 C[3][5]: 26 R[3][5]: 5
W[4][6]: 21 C[4][6]: 32 R[4][6]: 6

W[0][3]: 41 C[0][3]: 79 R[0][3]: 2
W[1][4]: 31 C[1][4]: 53 R[1][4]: 3
W[2][5]: 30 C[2][5]: 56 R[2][5]: 3
W[3][6]: 27 C[3][6]: 53 R[3][6]: 6

W[0][4]: 46 C[0][4]: 96 R[0][4]: 3
W[1][5]: 39 C[1][5]: 78 R[1][5]: 3
W[2][6]: 40 C[2][6]: 89 R[2][6]: 4

W[0][5]: 54 C[0][5]: 121    R[0][5]: 3
W[1][6]: 49 C[1][6]: 115    R[1][6]: 3

W[0][6]: 64 C[0][6]: 158    R[0][6]: 3

Minimum cost = 158
Root  = 3
```